

Game Server Portfolio

PolRob + CanvaSync

신입 게임 서버 개발자 지원용 프로젝트 포트폴리오

작성일: 2026. 06. 25

지원자: Shinwoo Seo / Contact: 이력서 참조

Positioning

실시간 게임 서버와 DB/클라우드 기반 실시간 협업 서비스를 함께 제시합니다.

핵심 요약

PolRob은 TCP/UDP, 방 단위 루프, 서버 권위 게임 규칙, headless bot 부하 테스트를 통해 게임 서버 역량을 보여줍니다. CanvaSync는 PostgreSQL, Redis, Azure Blob Storage, Azure 배포 경험과 DB 무결성/인덱스 개선으로 서버 백엔드 역량을 보완합니다.

C# / .NET

ASP.NET Core

SignalR

TCP / UDP

PostgreSQL / JSONB

Redis

Azure

Load Test

Portfolio Strategy

- PolRob은 게임 서버 메인 프로젝트로 배치했습니다. 네트워크, concurrency, 서버 권위, 부하 측정이 중심입니다.
- CanvaSync는 DB/클라우드/실시간 협업 보조 프로젝트로 배치했습니다. 졸업작품 최우수상과 Azure 전시 배포 경험을 함께 제시합니다.
- 대규모 상용 운영 경험으로 과장하지 않고, 직접 검증한 코드/배포/측정 결과만 근거로 삼았습니다.

1. 지원 직무와 프로젝트 매핑

모바일 게임 서버 개발 공고 기준으로 두 프로젝트의 증거를 역할별로 나눴습니다.

PolRob - 게임 서버 역량

- 6인 비대칭 실시간 추격 게임 서버.
- HTTP, SignalR, raw TCP, UDP를 역할별로 분리.
- 클라이언트는 조이스틱 입력만 전송하고 서버가 이동/충돌/체포/탈옥/승패를 판정.
- 방별 bounded command queue와 single-consumer loop로 상태 변경 순서를 제어.
- headless bot으로 60/300/600/900 bots 부하 테스트와 최적화 비교.

CanvaSync - DB/클라우드 역량

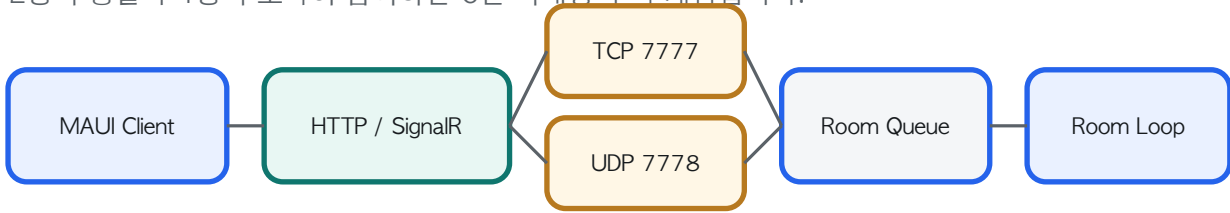
- 실시간 PDF 필기 동기화 웹 서비스.
- PostgreSQL, JSONB, Redis/InMemory, Azure Blob Storage 사용.
- Azure Cloud에 배포해 졸업작품 전시 환경에서 외부 사용자 사용 검증.
- 졸업작품 최우수상 수상.
- unique index, foreign key, query-plan 문서로 DB 설계 증거 보강.

공고 키워드 대응

요구 역량	증거 프로젝트	설명
모바일 게임서버 개발	PolRob	인게임 TCP/UDP, 방 루프, 서버 권위 게임 규칙
컨텐츠 데이터/DB	CanvaSync	PostgreSQL schema, JSONB, index, FK, EXPLAIN 문서
라이브 유지보수/트러블슈팅	PolRob + CanvaSync	부하 테스트 리포트, Azure 전시 배포, 장애/성능 개선 기록
네트워크/비동기	PolRob	SignalR, TCP/UDP, Channel<T>, ConcurrentDictionary, rate limit
클라우드 환경 이해	CanvaSync	Azure 배포, Blob Storage, PostgreSQL/Redis 연결 구성

2. PolRob - 실시간 모바일 게임 서버

2명의 경찰과 4명의 도둑이 참여하는 6인 비대칭 추격 게임입니다.



역할과 설계 목표

- 개인 프로젝트로 모바일 클라이언트, ASP.NET 서버, raw TCP/UDP 네트워크 서버, 부하 테스트 봇을 직접 구현했습니다.
- 신뢰성이 필요한 로비/인증/이벤트와 고빈도 이동 데이터를 분리해 HTTP, SignalR, TCP, UDP를 각각 사용했습니다.
- 게임 규칙은 클라이언트 좌표를 신뢰하지 않고 서버에서 계산하도록 이동/충돌/체포/탈옥/승패 판정을 서버로 옮겼습니다.

주요 구현 파일

polrob.Server/Network/GameNetworkServer*.cs

polrob.Server/Network/GameSession.cs

polrob.Shared/Models/PlayerMovementInput.cs

polrob.Test/BotRunner.cs

docs/server_optimization_report.md

3. PolRob - 문제 해결 사례

게임 서버 개발자로 보여주고 싶은 핵심 구현을 AS-IS / TO-BE 형식으로 정리했습니다.

Case 1. 클라이언트 좌표 신뢰 제거

AS-IS: 클라이언트가 보낸 위치를 그대로 믿으면 조작, 순간이동, 충돌 무시 문제가 생길 수 있습니다.

TO-BE: 클라이언트는 input vector와 sequence만 보내고, 서버가 속도/반지름/맵 충돌/시야/체포 판정을 계산합니다.

검증: movement session token, UDP endpoint, sequence, NaN/Infinity, 맵 경계를 검사합니다.

Case 2. 방 상태 동시 변경 문제

AS-IS: TCP join/leave, UDP move, game rule tick이 동시에 상태를 바꾸면 복합 규칙 순서가 흐려집니다.

TO-BE: 네트워크 수신부는 RoomCommand만 기록하고, 방별 single-consumer loop가 drain, simulation, rule tick, state sync를 처리합니다.

결과: 방 단위로 상태 변경 순서를 격리하고, 이동 입력은 같은 플레이어의 최신 입력으로 coalescing합니다.

Case 3. 과부하와 비정상 입력 방어

방 command queue는 bounded channel로 제한하고, UDP 입력에는 token-bucket rate limit을 적용했습니다. metrics 로그에 rate-limited, invalid, duplicate/late packet, queue length, dropped command를 노출해 부하 상황을 확인할 수 있게 했습니다.

4. PolRob - 부하 테스트와 최적화

UI 시뮬레이터 대신 실제 프로토콜을 사용하는 headless bot으로 서버 병목을 측정했습니다.

900

max bots

150 rooms 기준 로컬 부하
테스트 구간

-78.9%

CPU

final optimized branch local
comparison

-49.6%

UDP bytes

900 bots 구간 전체 UDP
bytes/s

-56.4%

Working Set

900 bots 구간 memory
footprint

측정 지표

- CPU, working set, GC allocation/pause, ThreadPool, lock contention
- UDP/TCP packets per second, UDP bytes per second, JSON serialization count
- connections, players, rooms, room phase, bot failures, eligible playing samples

최적화 내용

Lightweight UDP payload

패킷당 bytes와 JSON/GC 비용 감소

Server send tick

입력마다 즉시 broadcast 대신 방 tick에서 최신 snapshot 전송

Input coalescing

같은 플레이어의 오래된 move command를 최신 입력으로 병합

Idle send-rate reduction

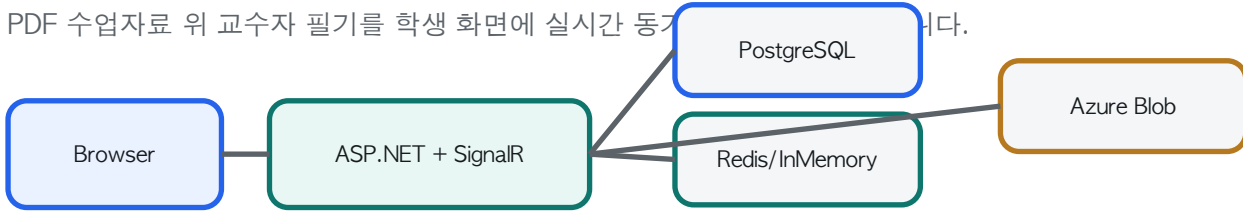
정지 상태의 불필요한 클라이언트 입력 송신 감소

검증 범위

수치는 동일 로컬 환경에서 baseline과 최적화 결과를 비교한 값입니다. 실제 상용 동시접속 수용량을 의미하지 않으며, 클라우드 VM, 네트워크 지연, packet loss는 별도 검증 대상입니다.

5. CanvaSync - 실시간 협업 서비스

PDF 수업자료 위 교수자 필기를 학생 화면에 실시간 동기화한다.



서비스 성격

- 교수자가 PDF 위에 작성한 도형/텍스트/펜 필기를 강의방 학생에게 실시간 전파.
- 학생은 교수자 필기와 분리된 개인 필기를 작성하고 병합 PDF를 다운로드.
- 졸업작품 전시 환경에서 Azure 배포 후 외부 사용자 접속 흐름 검증.
- 졸업작품 최우수상 수상.

서버 관점의 가치

- SignalR + MessagePack 기반 실시간 이벤트 동기화.
- PostgreSQL은 계정/강의/필기 snapshot 영속화.
- Redis/InMemory는 진행 중 필기 상태와 연결 정보를 저장.
- Azure Blob Storage에 PDF 원본 저장, DB에는 주소/메타데이터 보관.

주요 구현 파일

canvasync/Hubs/CanvasHub.cs

canvasync/Services/CanvasService.cs

canvasync/Data/CanvasDbContext.cs

canvasync/Controllers/PDFImagesController.cs

docs/database-design-and-tuning.md

6. CanvaSync - DB 설계와 튜닝 근거

단순 CRUD가 아니라 무결성, 접근 패턴, 조회 성능을 코드와 migration으로 보강했습니다.

DB 항목	상태	설명
Members.Name unique	적용	로그인/회원가입 조회 기준을 DB가 보장
Lectures.Code unique	적용	6자리 입장 코드 충돌 방지와 재시도 로직
DrawingData (LectureId, MemberId)	적용	강의별 사용자 필기 snapshot 중복 저장 방지
Foreign key + cascade	적용	강의/회원 삭제 시 orphan drawing data 방지
JSONB	적용	필기 요소를 페이지/사용자 단위로 통째로 로드하는 접근 패턴에 맞춤
EXPLAIN script	적용	주요 조회가 index scan을 타는지 확인하는 문서와 SQL 제공

저장 흐름

- 실시간 편집 중에는 Redis/InMemory 저장소에서 최신 drawing state를 조회합니다.
- 교수자 연결 종료 시 현재 drawing state를 PostgreSQL DrawingData에 snapshot으로 저장합니다.
- 저장 경로는 ExecuteUpdateAsync()를 먼저 시도하고, insert race는 unique conflict 후 update로 회복합니다.
- Controller와 Hub는 인증된 claim 기준으로 lecture 접근 권한을 확인합니다.

검증 결과

2026.06.25 기준 CanvaSync solution build는 0 warning / 0 error로 통과했습니다. DB 개선은 migration과 model snapshot에 반영되어 있습니다.

7. 채용 공고 기준 적합도와 보완 계획

MLB 라이벌 서버 프로그래머 공고의 요구사항을 프로젝트 증거와 연결했습니다.

공고 키워드	증거 수준	프로젝트 근거
모바일 게임서버	핵심 증거	PolRob: 모바일 클라이언트 + 인게임 서버 + TCP/UDP transport
컨텐츠 데이터 설계	보완 증거	CanvaSync DB 설계로 SQL/RDB 설계 역량 제시. 게임 master data는 추가 보완 가능
라이브 유지보수/자동화	개선 기록	부하 테스트/ 문서/ 배포 경험을 통해 문제 측정과 개선 과정을 제시
DB/ 쿼리/ 튜닝	보완 증거	PostgreSQL index, FK, JSONB, EXPLAIN 문서. MySQL은 동일 SQL/RDB 관점으로 설명
네트워크/비동기	핵심 증거	SignalR, TCP/UDP, Channel(T), room loop, rate limit
클라우드	배포 경험	CanvaSync Azure 전시 배포. 상용 대규모 운영 경험과는 구분해서 설명
PHP/Golang	학습 예정	주력은 C#/.NET. 지원 스택 적응을 위해 Go/PHP 소규모 API 과제 추가 예정
Docker/Linux	보완 예정	로컬 인프라 명령 경험을 서비스 컨테이너 실행 문서로 확장 예정

- 지원 전 마지막 보완
- PolRob 문서에서 미완성 표현(TBD 등)을 정리하고, 최종 combined benchmark를 한 번 더 재측정합니다.
 - CanvaSync EXPLAIN 결과를 실제 캡처해 DB 튜닝 문서에 붙입니다.
 - Docker/Linux 실행 경로와 Go 또는 PHP 소규모 도구를 하나 추가하면 공고 스택 미스매치를 줄일 수 있습니다.

8. 검증 스냅샷과 면접 포인트

포트폴리오에서 말할 때는 구현 사실과 검증 범위를 분리해서 설명합니다.

빌드 검증

- PolRob Server: dotnet build polrob.Server.csproj --no-restore -> 0 warning / 0 error
- PolRob Bot Test: dotnet build polrob.Test.csproj --no-restore -> 0 warning / 0 error
- CanvaSync Solution: dotnet build canvasync.sln --no-restore -> 0 warning / 0 error

면접에서 강조할 이야기

- PolRob: 왜 TCP와 UDP를 나눴는지, 왜 클라이언트 좌표를 신뢰하지 않는지, 방별 루프가 어떤 동시성 문제를 줄이는지 설명합니다.
- PolRob: 최적화 수치는 처리량 자량이 아니라 bottleneck을 분리해 측정하고 개선한 경험으로 설명합니다.
- CanvaSync: 왜 PDF 원본은 Blob Storage로 빼고, 필기 snapshot은 PostgreSQL JSONB로 저장했는지 설명합니다.
- CanvaSync: unique index, FK, 권한 검증, query-plan 확인을 통해 DB 무결성과 조회 패턴을 의식했다고 설명합니다.

검증 범위와 태도

두 프로젝트는 상용 대규모 서비스 운영 경험이 아니라 개인/졸업작품 기반 검증입니다. 따라서 실제 서비스 규모를 과장하지 않고, 코드와 측정 결과로 증명 가능한 범위를 명확히 제시합니다. 면접에서는 설계 판단, 디버깅 과정, 성능 측정 방식, 보안 검토 기준을 본인의 언어로 설명하는 데 집중합니다.